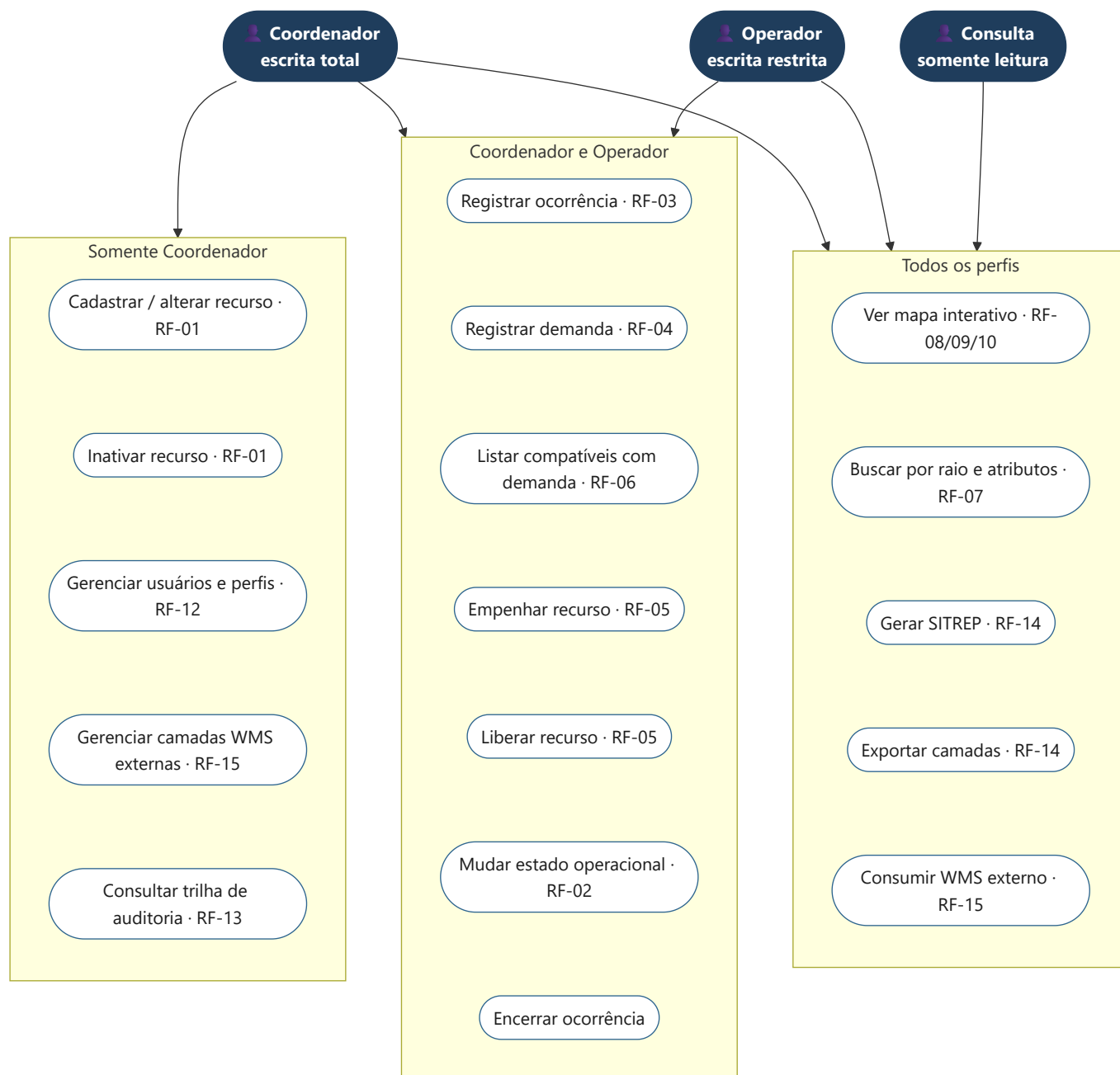


Diagramas UML

Diagrama de casos de uso

Mermaid não tem um tipo nativo de diagrama de casos de uso UML; a notação abaixo usa um fluxograma. Os casos de uso estão agrupados por **quem pode executá-los**: a seta de um ator para um grupo significa que ele executa todos os casos de uso daquele grupo.



Atores

Ator	Descrição
Coordenador	escrita total (RF-12): cadastro mestre de recursos, gestão de usuários, auditoria, e tudo que o Operador faz
Operador	escrita restrita às operações de campo: ocorrências, demandas, empenho/liberação, mudança de estado operacional

Ator	Descrição
Consulta	somente leitura: mapa, buscas, relatórios — nenhuma ação de escrita

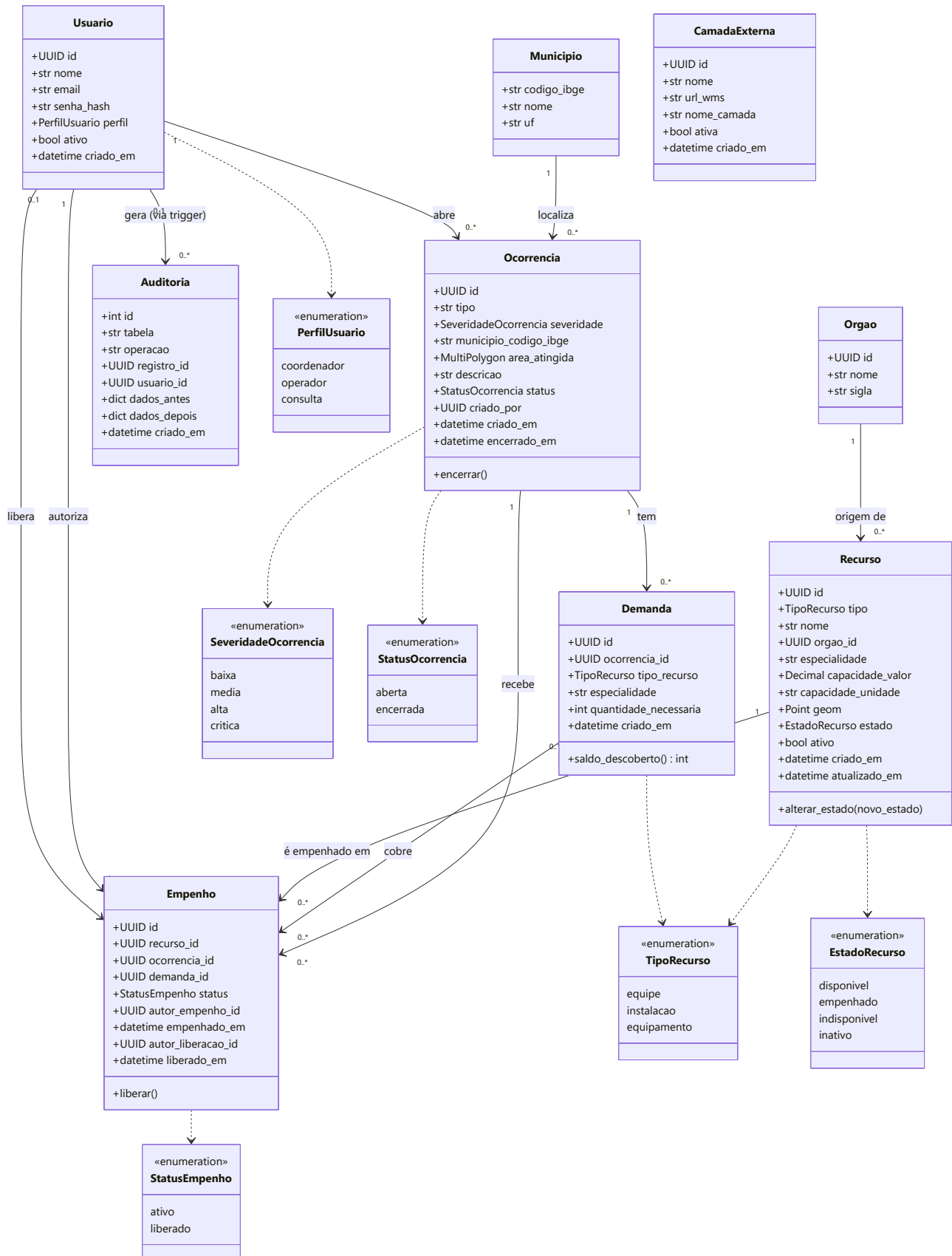
Caso de uso principal: empenhar recurso (CEN-1)

1. Operador (ou Coordenador) abre uma ocorrência com demanda descoberta.
2. Sistema lista recursos disponíveis e compatíveis, ordenados por distância (RF-06).
3. Usuário escolhe um recurso e confirma o empenho.
4. Sistema registra o empenho (autor, data/hora), muda o estado do recurso para "empenhado" e atualiza o saldo da demanda — tudo em uma única transação (RF-05).
5. Em até 5 interações a partir da tela de demandas (RNF-05) e refletido em qualquer outra sessão conectada em até 60s (RF-10, OBJ-2).

Este é o fluxo detalhado, passo a passo técnico, em sequencia-empenho.md.

Diagrama de classes

Reflete os modelos SQLAlchemy em `backend/app/models/`, que espelham o schema do banco. Este diagrama e o diagrama ER são gerados da **mesma definição do modelo**: cada classe corresponde a uma tabela, com os mesmos atributos, relacionamentos e cardinalidades (tabela de correspondência em `modelo-dados.md`).



Notas de desenho

- `Recurso.alterar_estado()` e `Ocorrencia.encerrar()` estão representados como métodos de domínio para legibilidade do diagrama, mas na implementação vivem na camada de serviço (`backend/app/services/`) — os modelos SQLAlchemy em si são anêmicos (armazenam estado, não comportamento). A regra que esses métodos protegem é reforçada de novo no banco (trigger + constraint), então a "lógica" real não depende de nenhuma dessas duas camadas isoladamente.
- `Auditoria` não aparece com métodos de escrita porque não tem nenhum — é populada só pelo trigger `fn_auditoria()` (ver modelo-dados.md), nunca pela camada de aplicação. O vínculo *Usuario gera Auditoria* é 0..1 porque `usuario_id` é uma referência **lógica** (sem constraint de chave estrangeira): o trigger grava a trilha mesmo quando a escrita não vem de um usuário autenticado.
- `Empenho` tem dois vínculos com `Usuario`: quem **autoriza** o empenho (sempre existe, 1) e quem **libera** (só existe depois da liberação, 0..1).
- Um `Empenho` é imutável quanto ao vínculo `Recurso ↔ Ocorrencia`: para mudar de recurso, libera-se o empenho atual e cria-se outro. Isso mantém o histórico (RF-05) sempre correto.

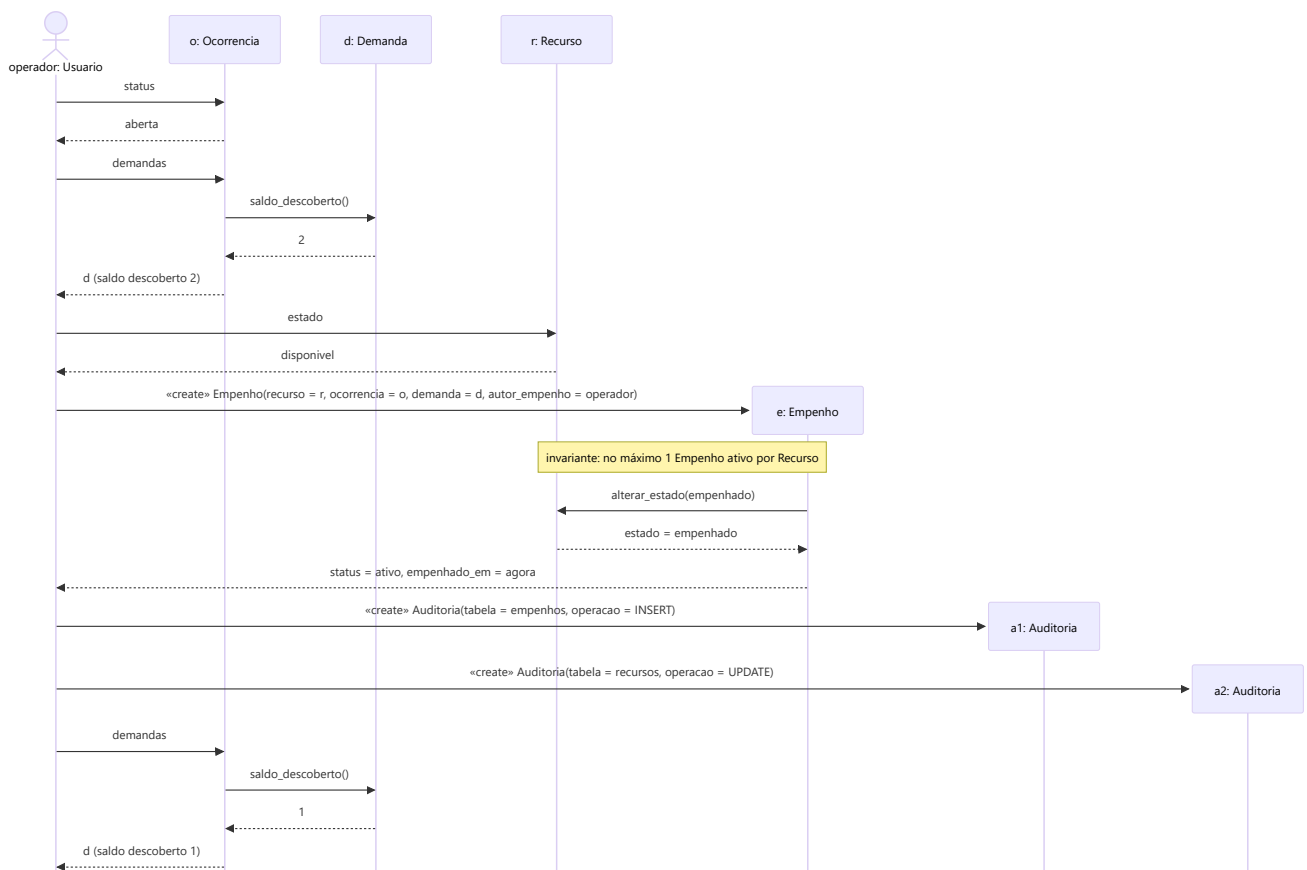
Diagramas de sequência

Cada linha de vida é um **objeto de uma classe do diagrama de classes** (`Usuario`, `Ocorrencia`, `Demanda`, `Recurso`, `Empenho`, `Auditoria`), no formato `objeto: Classe`. As mensagens são operações dessas classes (`saldo_descoberto()`, `alterar_estado()`, `liberar()`, criação de objeto) e só percorrem associações que existem no diagrama de classes:

Mensagem	Associação usada (diagrama de classes)
operador cria («create») e: Empenho	Usuario autoriza Empenho (1 : 0..*)
o consulta d.saldo_descoberto()	Ocorrencia tem Demanda (1 : 0..*)
e chama r.alterar_estado()	Recurso é empenhado em Empenho (1 : 0..*)
e se vincula a o e a d	Ocorrencia recebe Empenho · Demanda cobre Empenho
operador gera («create») a1, a2: Auditoria	Usuario gera Auditoria (0..1 : 0..*)

1. Empenhar um recurso (RF-05)

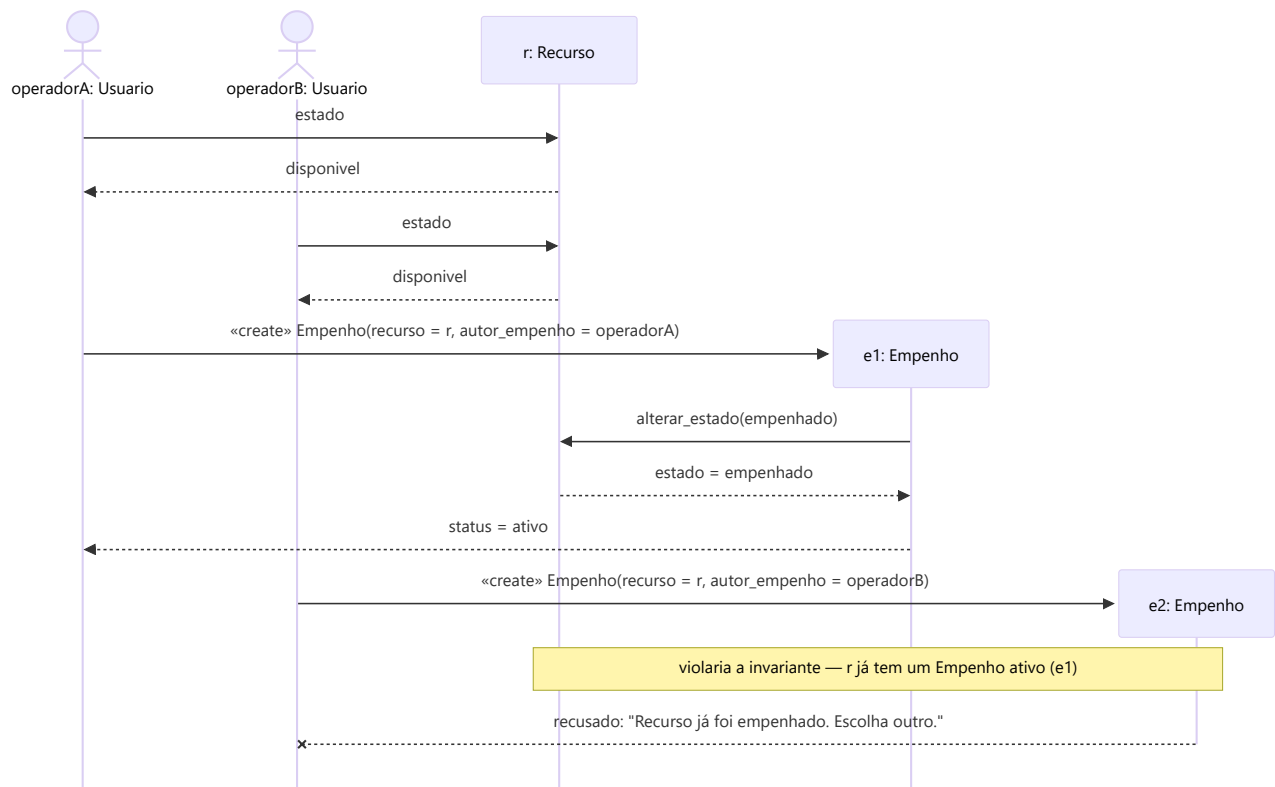
O operador é um `Usuario` com perfil `operador` (o ator do caso de uso *Empenhar recurso*).



Leitura: o operador confere que a ocorrência está aberta, que a demanda ainda tem saldo descoberto e que o recurso está disponível; cria o `Empenho`, que leva o `Recurso` para `empenhado`; cada escrita gera um registro de `Auditoria` (`a1`, `a2`) em nome do operador; o saldo descoberto da demanda cai de 2 para 1.

2. Duplo empenho concorrente (OBJ-3)

Dois operadores tentam empenhar o **mesmo** recurso quase ao mesmo tempo.



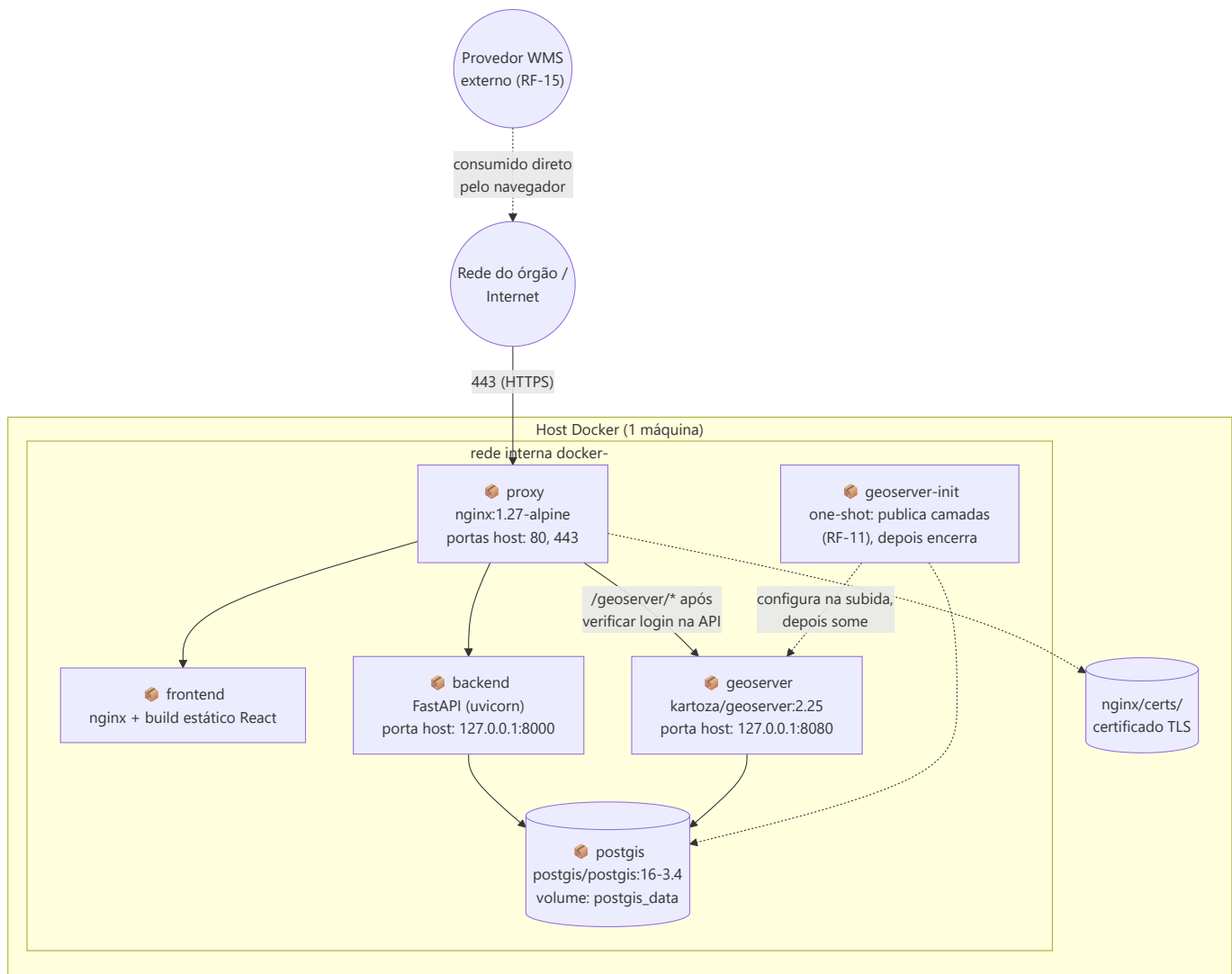
As duas leituras de `estado` devolvem `disponivel`, mas quem decide é a **invariante** "no máximo 1 `Empenho` ativo por `Recurso`", verificada no momento da criação do empenho — não a leitura anterior. Por isso `e2` nunca chega a existir e o operador B recebe a mensagem com a ação corretiva (RNF-05).

Como as operações são garantidas na implementação

Operação / regra (diagramas)	Onde é garantida
criar <code>Empenho</code>	<code>services/empenhos.py::empenhar_recurso</code> (POST <code>/api/empenhos</code>)
invariante "1 <code>Empenho</code> ativo por <code>Recurso</code> "	índice único parcial <code>uq_empenho_ativo_por_recurso</code> no banco
<code>Recurso.alterar_estado()</code>	serviço + trigger <code>trg_recursos_valida_estado</code> (rejeita transição inválida)
<code>Demanda.saldo_descoberto()</code>	<code>services/ocorrencias.py::_demanda_para_out</code>
gerar <code>Auditoria</code>	trigger <code>fn_auditoria()</code> em cada escrita, com o usuário da sessão

Diagrama de implantação

Topologia dos containers Docker Compose (`docker-compose.yml`) numa única máquina (o RNF-07 pede implantação em uma máquina limpa em até 60 minutos — não há dependência de infraestrutura distribuída).



Requisitos de máquina

Item	Mínimo recomendado
CPU	2 vCPU
RAM	4 GB (GeoServer sozinho pede ~1 GB)
Disco	10 GB livres (volumes crescem com o histórico de auditoria e os dados espaciais)
Software	Docker Engine + Docker Compose plugin
Rede	portas 80 e 443 livres; 8000 e 8080 livres no loopback (<code>127.0.0.1</code>) — usadas só para diagnóstico e administração local, nunca expostas à rede (RNF-06)

Por que um `geoserver-init` que "aparece e some"

GeoServer não vem com workspace/datastore/camadas pré-configurados — isso normalmente é um passo manual no console web. `geoserver-init` (`geoserver/init_geoserver.sh`) é um container *one-shot*: sobe depois que `geoserver` e `backend` estão saudáveis, chama a REST API do GeoServer para criar o workspace `siscoord`, o

datastore PostGIS e publicar as camadas `recursos / ocorrencias`, e termina. É reexecutável sem efeito colateral (idempotente) — rodar `docker compose up` de novo não duplica nada.

Passo a passo de implantação

Ver `manual-instalacao.md` para o procedimento completo, escrito para alguém que nunca viu o projeto (RNF-07).